

SMARTJOURNEY

An AI-Powered Multi-Agent Trip Planner for Sri Lanka

Final Project Report

Group 08

230247T Hewawasam H.H.S.T.T.

230279R Jayasinghe E.A.T.D.

Mentor: Mr. Isuru Ariyaratna

Live system: <https://aismartjourney.vercel.app>

October 2026

Declaration

We declare that this report is our own work and that the system described was designed and built by the members of Group 08. Sources of information used are acknowledged in the references.

Acknowledgement

We thank our mentor, Mr. Isuru Ariyaratna, for his guidance throughout the project, and the volunteers who took part in the usability study.

Abstract

Planning a trip to Sri Lanka requires combining information from many separate sources, and general-purpose AI chatbots can suggest places that do not exist or plans that exceed the traveller's budget. This report presents SmartJourney, a web-based, AI-powered, multi-agent trip planner that generates day-by-day itineraries from natural-language requests using verified local data. The system has a Next.js web application, a NestJS API that is the only trust boundary, and a FastAPI service that runs a LangGraph planning pipeline with Gemini and Groq failover, an output validator and a deterministic fallback planner. A retrieval-augmented knowledge base answers destination questions with citations and refuses when evidence is weak. Scheduled connectors keep about 6,500 listings in 25 districts current, and an administrator reviews imported data before it goes live. The system is deployed on Vercel and AWS with a CI/CD pipeline that gates releases on automated tests. Testing covers unit, end-to-end, browser-journey, golden-scenario, load and retrieval checks, and a System Usability Scale study with external users evaluates usability. The report describes the design, implementation, results and limitations, and proposes future work.

Keywords: trip planning, multi-agent systems, large language models, retrieval-augmented generation, LangGraph, usability, Sri Lanka.

Table of Contents

1. Introduction	6
1.1 Background.....	6
1.2 Problem Statement.....	6
1.3 Aim and Objectives	6
1.4 Scope	7
1.5 Contributions	7
1.6 Organisation of the Report	7
2. Literature Survey	9
2.1 Existing Travel-Planning Systems.....	9
2.2 Large Language Model Agents	9
2.3 Retrieval-Augmented Generation.....	9
2.4 Orchestration with LangGraph	10
2.5 Web and Data Technologies.....	10
2.6 Usability Evaluation	10
2.7 Research Gap.....	10
3. Methodology.....	11
3.1 Development Approach	11
3.2 Requirements and Design Process.....	11
3.3 Technology Selection	11
3.4 Data Sources and Ethics	12
3.5 Evaluation Methodology	12
4. System Design and Architecture	13
4.1 Architectural Overview	13
4.2 Component Responsibilities.....	14
4.3 The Planning Pipeline.....	14
4.4 Follow-up Editing.....	16
4.5 LLM Failover and Fallback.....	16
4.6 Data Design	16
4.7 Security Design.....	17
4.8 Deployment Architecture.....	17
5. Implementation.....	18
5.1 Web Front End.....	18
5.2 NestJS API.....	19
5.3 AI Service.....	20
5.4 Retrieval-Augmented Question Answering.....	20
5.5 Data Ingestion and Refresh.....	21
5.6 Administration	22
5.7 Continuous Integration and Delivery.....	22
6. Testing and Evaluation	23
6.1 Test Strategy	23

6.2 Unit and Integration Results	23
6.3 Security and Access-Control Testing	24
6.4 Golden Scenario Testing	24
6.5 RAG Evaluation	24
6.6 Performance and Load.....	25
6.7 Defects and Limitations Found.....	25
6.8 Accessibility and Browser Checks	26
7. Usability Evaluation	27
7.1 Method.....	27
7.2 Participants	27
7.3 SUS Results	27
7.4 Results by Statement	28
7.5 Discussion.....	29
8. Results and Discussion	31
8.1 Achievement of Objectives.....	31
8.2 Strengths	31
8.3 Limitations.....	31
8.4 Threats to Validity.....	32
9. Conclusion and Future Work.....	33
9.1 Conclusion.....	33
9.2 Future Work.....	33
References	34
Appendix A: Deliverables and Where to Find Them	36
Appendix B: Automated Test Inventory	37
Appendix C: Usability Questionnaire.....	38
Appendix D: Team Contributions	39

1. Introduction

1.1 Background

Sri Lanka receives visitors who want to combine cultural sites, beaches, hill-country scenery and wildlife in a short time. Planning such a trip is difficult because the information needed is spread over many unrelated sources: accommodation sites, review platforms, map applications, weather services and local advice. A traveller must decide which places to visit, in what order, how to move between them and how much the whole trip will cost, and must keep these decisions consistent with each other.

Recent large language models (LLMs) can hold a conversation about travel, but a general-purpose chatbot has no access to a verified, destination-specific database. It can therefore suggest places that are closed, mislocated or invented, and it cannot guarantee that a plan fits a stated budget. SmartJourney was built to address this gap. It combines a conversational interface with a multi-agent AI planner that works only from verified local data, so the answer is both personalised and trustworthy.

1.2 Problem Statement

Travellers planning a trip to Sri Lanka face four connected problems: (i) information is fragmented across many platforms; (ii) general-purpose AI tools produce unverified and sometimes impossible suggestions; (iii) balancing time, budget and personal interests is a logistics problem that is slow to solve by hand; and (iv) plans must adapt to real conditions such as weather, natural-disaster alerts and public holidays. No single tool available to the target users solves these problems together.

1.3 Aim and Objectives

The aim of the project is to design, implement, deploy and evaluate a web-based, AI-powered, multi-agent trip-planning platform for Sri Lanka. The specific objectives, taken from the project proposal, are:

- To deliver a functional, centralised smart-tourism platform with a web application, an administration panel and supporting services.
- To analyse the travel-planning challenges faced by travellers in Sri Lanka.
- To implement personalised itinerary generation from natural-language requests.

- To develop an agentic AI system for goal-oriented trip management that relies on curated, verified data.
- To provide budget planning, tracking and optimisation.
- To evaluate the system through automated testing and a usability study with external users.

1.4 Scope

In scope. User registration and sign-in; conversational trip planning; day-by-day itineraries with costs and a map; targeted follow-up edits; saved trips; a budget tracker; an explore section for districts, listings and events; email notifications for budget alerts; an administrator interface for reviewing data, managing users, viewing analytics and configuring AI models; a retrieval-augmented question-answering feature; automated data collection; cloud deployment with continuous integration and delivery.

Out of scope. Subscription billing was not built. Restaurant photographs are replaced by a placeholder because no free image source was available.

1.5 Contributions

- A four-service architecture in which a single NestJS service is the only trust boundary between the browser and the AI service.
- A LangGraph-based planning pipeline in which the LLM interprets requests while the verified database supplies all facts, followed by output validation and a deterministic fallback planner.
- A retrieval-augmented generation (RAG) knowledge base with hybrid search, citations and a refusal gate.
- An automated, scheduled data-ingestion framework with an admin-review step before data goes live.
- A layered test strategy covering unit, end-to-end, browser-journey, golden-scenario, load and retrieval-quality checks, with CI/CD gating of deployments.
- A live deployment on Vercel and AWS and a structured SUS usability study.

1.6 Organisation of the Report

Chapter 2 surveys existing systems and the technologies used. Chapter 3 describes the methodology. Chapter 4 presents the system design, and Chapter 5 the

implementation. Chapter 6 reports testing, Chapter 7 the usability evaluation, and Chapter 8 discusses the results against the objectives. Chapter 9 concludes and outlines future work.

2. Literature Survey

2.1 Existing Travel-Planning Systems

Several mature products address parts of the travel-planning problem. Table I summarises those reviewed in the project proposal and feasibility study [1]–[3].

TABLE I
COMPARISON OF EXISTING SYSTEMS

System	Strength	Limitation for the target problem
Google Travel [1]	Strong maps, itinerary and booking links	Global, general data; limited local curation; no conversational budget-aware planning for Sri Lanka
TripAdvisor [2]	Large volume of crowd-sourced reviews	No automated, personalised itinerary generation; reviews are unverified
Booking.com [3]	Reservations and prices	Itinerary planning is minimal
General chatbots (e.g., Gemini, ChatGPT, Claude)	Natural conversation and flexible advice	No verified local database; may invent places; no guaranteed budget compliance
SmartJourney (this work)	Conversational planning on verified local data, budget tracking, weather and disaster awareness, cited Q&A	Free-tier AI quotas

The comparison shows that the unfilled niche is a conversational planner that is grounded in a verified local database and that validates its own output.

2.2 Large Language Model Agents

LLM agents extend a language model with tools and a control loop so that it can gather information and act. The ReAct approach interleaves reasoning steps with tool calls and observations, which improves reliability on tasks that need external facts [4]. SmartJourney follows this idea: its planner and recommendation agents call tools for weather, disasters, geocoding, routing and database search. Because a model can fail or hallucinate, the system treats the model's output as a proposal that must pass deterministic validation before it reaches the user.

2.3 Retrieval-Augmented Generation

Retrieval-augmented generation grounds a model's answer in retrieved documents rather than only its trained parameters [5]. A typical pipeline splits documents into chunks, converts them to vectors with an embedding model, and retrieves the chunks closest to the question. Dense vector search alone can miss exact keywords, so hybrid

retrieval combines it with keyword search. Reciprocal rank fusion (RRF) is a simple, parameter-light way to merge ranked lists from different retrievers and has been shown to outperform individual rankers [6]. SmartJourney uses hybrid retrieval with RRF, a minimum-score gate and mandatory citations.

2.4 Orchestration with LangGraph

LangGraph represents an agent workflow as a graph of nodes that read and update a shared state, with conditional edges between them [7]. This suits trip planning, where steps such as validation, slot filling, weather checking and verification must run in a defined order and where a failure should route to a repair or fallback node instead of ending the request.

2.5 Web and Data Technologies

The platform uses Next.js for the web front end [8], NestJS for the API layer [9], FastAPI for the AI service [10], PostgreSQL with the PostGIS and pgvector extensions for relational, geographic and vector data [11], [12], and Keycloak for identity and access management [13]. Prisma is used as a type-safe database client [14], and the Gemini API is the primary language model [15]. These were chosen because they are widely adopted, well documented and support the separation of concerns that the architecture requires.

2.6 Usability Evaluation

The System Usability Scale (SUS) is a ten-item questionnaire that yields a single score from 0 to 100 and is widely used as a quick, reliable usability measure [16]. A mean score of about 68 is commonly treated as the average benchmark, and scores above about 80 are regarded as excellent [17]. SUS is used in this project because it is inexpensive, standardised and comparable with other systems. Its author later reviewed how the scale has been used in practice [18].

2.7 Research Gap

Existing products either provide data without planning (booking and review sites) or conversation without verified data (general chatbots). The gap addressed here is a planner that is conversational, grounded in verified Sri Lankan data, budget-aware, safe under failure, and evaluated with real users.

3. Methodology

3.1 Development Approach

The project followed an iterative, incremental approach. Requirements were captured in the proposal, feasibility study and Software Requirements Specification (SRS), and the architecture in the design document. Work then proceeded in short cycles: build a vertical slice of a feature across the three services, test it, record decisions, and refine. Each repository used Git with feature branches and pull requests, and continuous integration ran the test suite on every push.

3.2 Requirements and Design Process

- **Problem analysis** of existing platforms and traveller needs (Chapter 2).
- **Requirements** recorded as functional and non-functional requirements in the SRS, grouped by traveller, administrator and AI-system actors.
- **Architecture** in which each service has one responsibility and the browser reaches only the NestJS API.
- **Decision log** for significant choices, for example D13: the database schema is owned by hand-written SQL migrations and Prisma only introspects it.

3.3 Technology Selection

TABLE II
TECHNOLOGY STACK

Layer	Technology	Reason for selection
Web front end	Next.js 14 (App Router), TypeScript, Tailwind CSS, Zustand, next-auth, Leaflet	Server and client rendering, typed code, fast UI development, open-source maps
API	NestJS, Prisma 7, Keycloak JWT guards	Modular structure, type-safe data access, standard token-based security
AI service	Python, FastAPI, LangGraph, LangChain tools	Python AI ecosystem; explicit graph control of the workflow
LLMs	Gemini (primary), Groq (failover), optional OpenAI and Anthropic models set by the admin, deterministic fallback planner	Free-tier access, resilience to quota and outage
Data	PostgreSQL, PostGIS, pgvector, Redis	Relational, geographic and vector search in one database; caching
Identity	Keycloak with traveller and admin roles	Proven identity provider, email verification, role-based access
Delivery	Docker Compose, GitHub Actions, GHCR, Vercel, AWS EC2, Caddy	Reproducible builds, gated deployment, automatic HTTPS

3.4 Data Sources and Ethics

Place data come from OpenStreetMap, Booking.com, Wikidata and administrator entries. Events and heritage-site entry fees come from scrapers and enter the live data only after administrator approval. Knowledge-base text comes from Wikivoyage and curated notes. External services are called through their public interfaces and within free-tier limits, with caching to reduce load. Usability participants are volunteers who give consent, answer anonymously and receive a pre-created test account, so no personal data beyond a participant ID is collected by the study form.

3.5 Evaluation Methodology

Quality was assessed on three levels. First, automated tests check each service and each boundary between services. Second, scripted golden scenarios and a golden question set check the behaviour of the non-deterministic AI parts by asserting properties such as schema validity and budget compliance rather than exact text. Third, a usability study with 20–25 external users completes a fixed task list and the SUS questionnaire (Chapter 7).

4. System Design and Architecture

4.1 Architectural Overview

SmartJourney is organised as four deployable parts: the Next.js web application (frontend-web), the NestJS API (backend), the FastAPI AI service (ai-backend) and the data stores. Fig. 1 shows how they connect. The browser communicates only with the Next.js application and the NestJS API. The NestJS API verifies the user's identity token, applies role and ownership rules, and forwards planning requests to the AI service with a shared secret. The AI service is reachable only inside the private network.

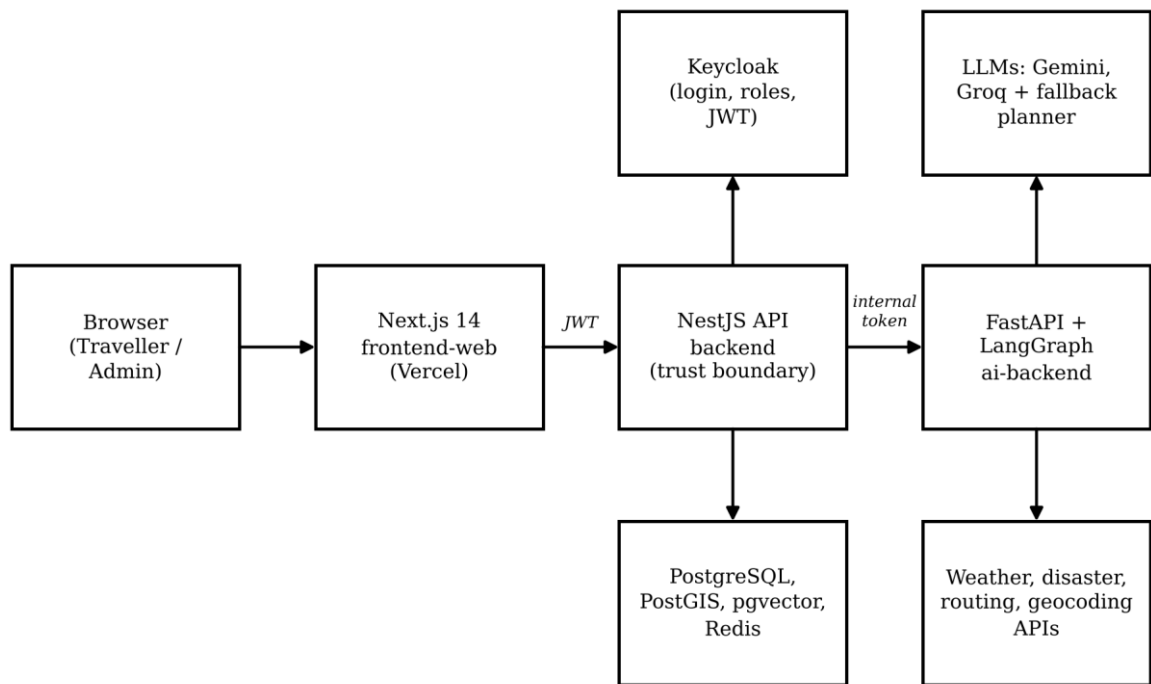


Fig. 1. High-level architecture of SmartJourney.

This design has one important security consequence. The AI service has no user authentication of its own, so the NestJS guards form the only trust boundary of the system. Network isolation and the internal token protect the AI service, and this is documented as a residual risk that depends on correct deployment.

4.2 Component Responsibilities

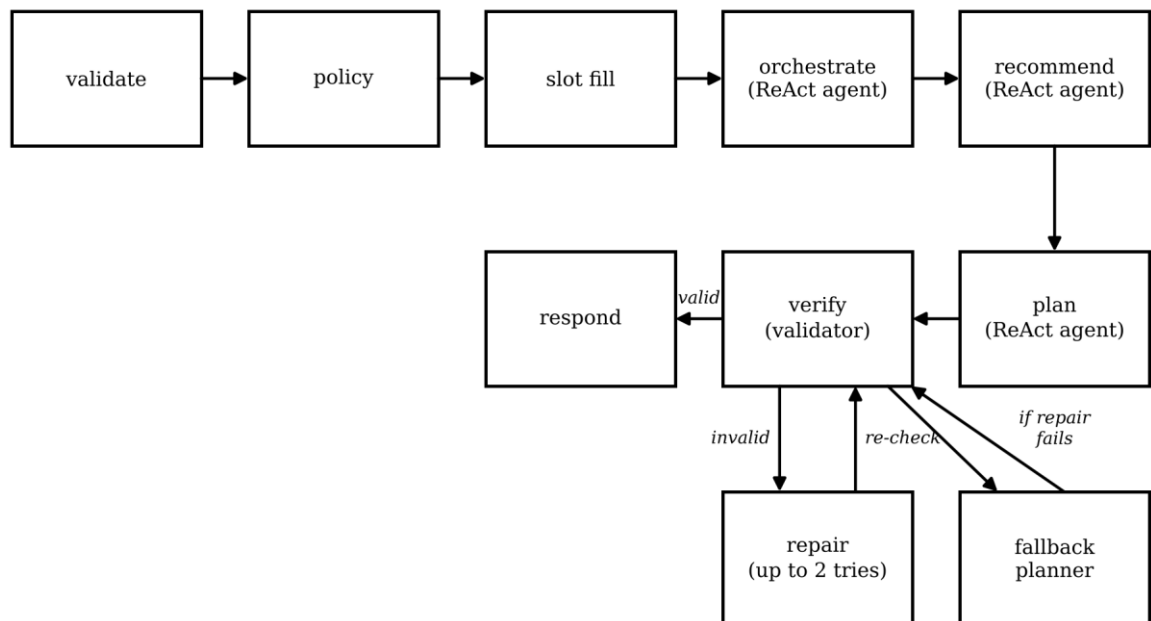
TABLE III
COMPONENTS AND RESPONSIBILITIES

Component	Responsibilities
frontend-web	Landing page, sign-in, chat and trip view with map, budget tracker, saved itineraries, explore pages, admin screens; keeps session and trip state; talks only to NestJS

Component	Responsibilities
backend (NestJS)	Authentication and role guards; users; chat; trips; budget; explore (districts, listings, events); admin (review, users, analytics, AI settings, sync triggers); proxy to the AI service; owner of the database schema
ai-backend (FastAPI)	Planning pipeline, agents and tools; scoring and clustering; budget logic; output validator; fallback planner; follow-up editing; RAG ingestion and answering; data connectors and scheduler
Keycloak	Login, email verification, token issuing, traveller and admin roles
PostgreSQL + PostGIS + pgvector	Users, itineraries, expenses, listings, events, sessions and knowledge chunks with vectors
Redis	Cache for routing and other repeated calls

4.3 The Planning Pipeline

The planner is a LangGraph state machine whose nodes read and update a shared trip state. A request enters at validation and passes through the policy check and slot filling. The orchestrate node is a ReAct agent that resolves the destination and district and calls tools for the calendar, weather and disaster alerts. The recommendation agent then scores and selects verified listings, and the planner agent assembles the days. A verification node checks the plan. If the plan is invalid, control loops through the repair node and back to verification; if repair fails, control passes to the fallback planner, whose plan is also checked by verification before the response is sent. Fig. 2 shows the main path and these loops.



Early exits go straight to respond (clarification needed, policy or validation error, destination not found). Weather questions end at a weather node; pure questions go to the RAG answer node. A recommend step with no results goes directly to the fallback.

Fig. 2. Main path of the LangGraph planning pipeline, with the verify–repair and verify–fallback loops.

- **Validate and policy.** Reject malformed or disallowed input before any model call.
- **Slot filling.** Extract destination, number of days, budget, interests and start location from the conversation, asking for missing items. If something is missing the graph goes straight to the response.
- **Orchestrate.** A ReAct agent resolves the place and calls the location, calendar, weather and disaster tools. If no district can be resolved, the graph asks for clarification instead of planning.
- **Recommend and plan.** Score and cluster verified listings by geography and interest, then assemble days within the budget. If nothing is recommended, the graph goes directly to the fallback planner.
- **Verify.** Check that the output is schema-valid, uses only known places, has coordinates inside Sri Lanka and respects the budget.
- **Repair loop.** An invalid plan is sent to the repair node, which first tries a deterministic rebuild of only the failing days and otherwise asks the model again at a higher temperature. The result returns to verification. At most two repair attempts are made, and the loop stops early if a repair reproduces the same failures as the one before it.
- **Fallback.** When repair is exhausted or makes no progress, the deterministic planner builds a valid plan, which again passes through verification so that the user never sees a server error.
- **Intents.** A pure question goes from slot filling to the RAG answer node; a request to plan and ask goes through the full pipeline and then to the answer node; a weather question is answered from the live forecast; a shape-only follow-up goes to a targeted re-plan and then to verification.

The principle that keeps the pipeline reliable is that the model understands the request but the database supplies the facts. The planner can only choose from verified listings, so it cannot invent a place.

4.4 Follow-up Editing

A follow-up such as "make day 2 cheaper" is classified and applied only to the targeted day. The other days are kept unchanged, which makes the interaction predictable and cheaper to compute. Informational follow-ups, such as asking about weather, are answered without rebuilding the plan.

4.5 LLM Failover and Fallback

Models are tried in an administrator-configured order, by default Gemini and then Groq; OpenAI and Anthropic models can also be configured by the administrator. Quota errors and timeouts move the request to the next model. If every model fails, the deterministic rule-based planner returns a plan marked `plan_source = fallback`. This design accepts that free-tier models are unreliable and makes the failure path a tested feature.

4.6 Data Design

The database holds relational data (users, itineraries, expenses, sessions), geographic data (listing coordinates and districts through PostGIS) and vector data (knowledge-chunk embeddings through pgvector). The schema is owned by 19 sequential SQL migration files (0000 to 0018) applied by a runner that records a SHA-256 checksum of each file, so a changed or skipped migration is detected. Prisma introspects the resulting schema for typed access (decision D13). Every query on user data includes the user identifier, and a request for another user's resource returns 404 so that its existence is not revealed.

4.7 Security Design

- **Authentication.** Keycloak issues RS256-signed JWTs; NestJS verifies signature, issuer and audience.
- **Authorisation.** Global JWT and role guards; admin routes require the admin role.
- **Data isolation.** Ownership scoping on every user query; 404 for foreign resources.
- **Service-to-service trust.** The AI service accepts planning calls only with the X-Internal-Token secret and is not exposed publicly.
- **Secrets.** API keys saved through the admin panel are encrypted; configuration is supplied through environment variables.
- **Account policy.** Email verification and a password policy of 8–12 characters with mixed case, a digit and a symbol.

4.8 Deployment Architecture

The web application is deployed on Vercel. The NestJS API, AI service, Keycloak, PostgreSQL and Redis run as Docker containers on one AWS EC2 instance in the Mumbai region, fronted by a Caddy reverse proxy that obtains HTTPS certificates

automatically. Only ports 80, 443 and SSH are open; the database, cache and AI service are reachable only inside the Docker network. Images are built by GitHub Actions, stored in the GitHub Container Registry and tagged with the commit SHA. A nightly script dumps both databases and keeps seven copies.

5. Implementation

5.1 Web Front End

The front end is a Next.js 14 application using the App Router and TypeScript. Route middleware gates pages by authentication and role. The next-auth integration with Keycloak handles sign-in and token refresh. A Zustand store keeps the current trip so the chat panel, day view, map and budget stay consistent. The main screens are the landing page, chat and itinerary, saved itineraries, budget tracker, explore, profile and the admin screens. The interface uses motion and a responsive layout, and the map uses Leaflet with open map tiles.

Figs. 3 to 6 show the main screens: the chat with the generated itinerary and route map, the budget tracker with its recent expenses, and the explore page.

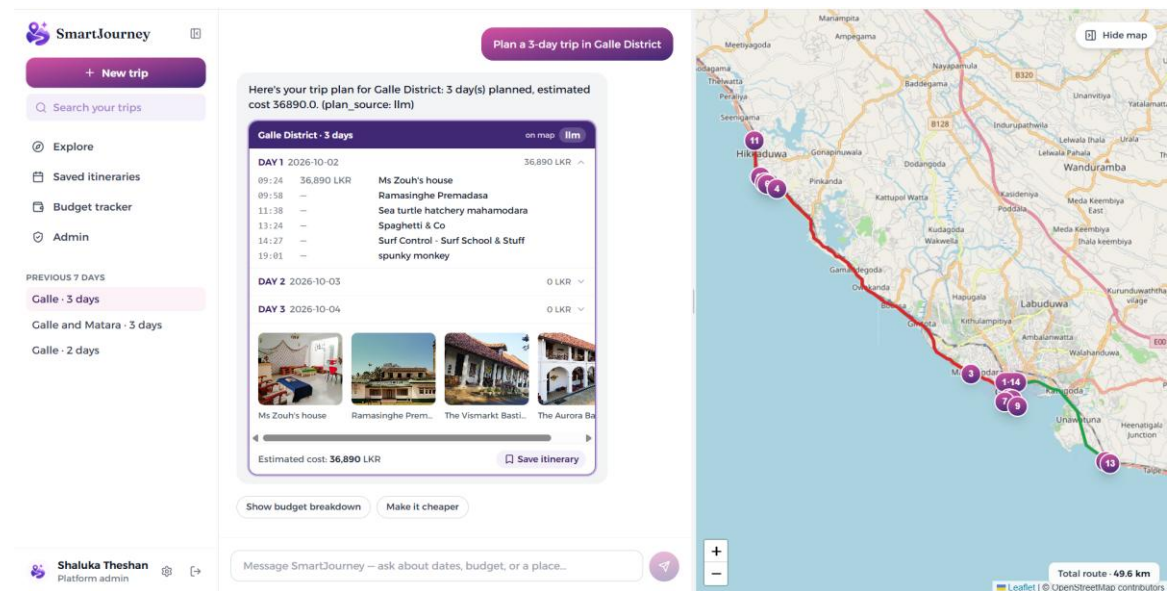


Fig. 3. Chat and itinerary screen with the route map.

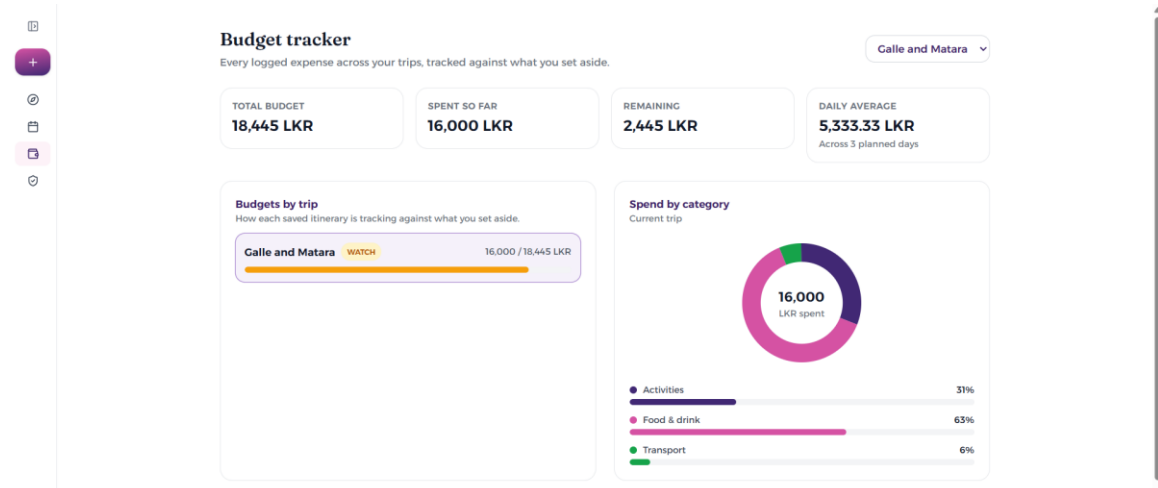


Fig. 4. Budget tracker showing budget by trip and spend by category.

Recent expenses
Logged against this trip.

+ Add expense

DATE	DESCRIPTION	CATEGORY	AMOUNT
10/1/2026	petrol	Transport	1,000 LKR
10/1/2026	lunch and dinner	Food & drink	10,000 LKR
10/1/2026	water park	Activities	5,000 LKR

Fig. 5. Recent expenses logged against a trip.

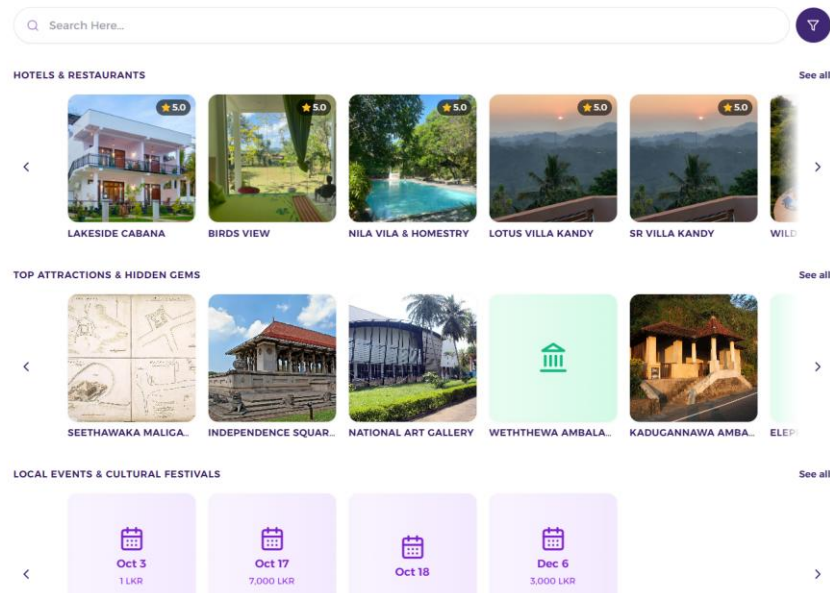


Fig. 6. Explore page with hotels, attractions and local events.

5.2 NestJS API

The API is divided into modules for authentication, users, trips, budget, chat, explore and administration, exposing about 65 route handlers. Services take the user identifier as their first parameter and apply it in every query. The budget module records expenses against a saved itinerary and computes a status from two thresholds, 80% and 100% of

the budget, giving on track, watch or over budget. A notification scheduler sends email alerts for budget status. The chat module forwards messages to the AI service through a client that maps timeouts, network errors and non-success responses to a 502 response instead of an unhandled exception. The admin module supports review and approval of imported listings and events, user management, analytics, AI-model configuration and manual triggers for data synchronisation.

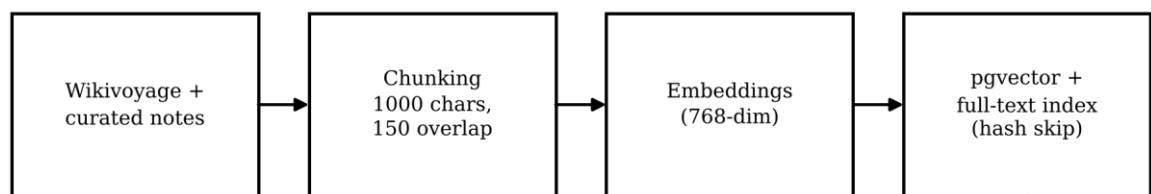
5.3 AI Service

The AI service is a FastAPI application whose planning endpoint runs the LangGraph pipeline described in Section 4.3. Prompts are kept in one central package, and a test fails if a prompt string appears elsewhere. The scoring, clustering, itinerary and budget modules are deterministic, so the same input always gives the same ranking and the same day plan. The output validator is the largest single test target because it protects users from incorrect model output. The routing tool uses a cache first, then OpenRouteService, and finally a haversine distance estimate, so travel times are always available.

5.4 Retrieval-Augmented Question Answering

The RAG feature answers questions about destinations from a knowledge base and shows its sources. Fig. 7 summarises the flow.

Ingestion (offline)



Query time

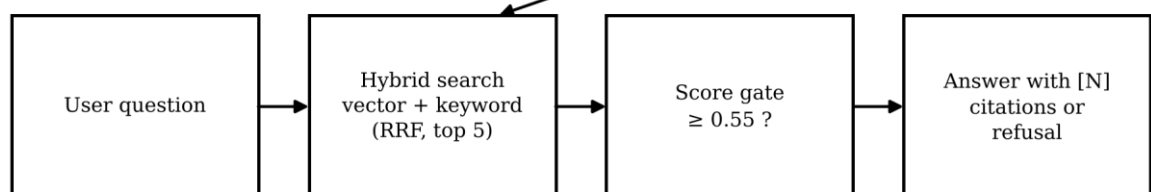


Fig. 7. RAG ingestion (top) and query-time answering (bottom).

TABLE IV
RAG DESIGN PARAMETERS

Stage	Design choice
Chunking	1000 characters with 150 overlap; section headings and a breadcrumb prefix kept with each chunk
Embeddings	Gemini embedding model, 768 dimensions, with retries for free-tier rate limits
Re-ingestion	A content hash lets unchanged documents be skipped
Retrieval	Vector search plus full-text search, merged by reciprocal rank fusion ($k = 60$), candidate pool of 20, top 5 returned
Refusal gate	If the best score is below 0.55 the system replies that it has no reliable information
Answer	Grounded in the retrieved chunks, with [N] citations; uncited answers are treated as failures

The feature is controlled by the `ENABLE_RAG` setting, which is off by default in the example configuration and must be switched on in the deployed environment.

5.5 Data Ingestion and Refresh

Data enter through connectors that implement a common interface: fetch from a source, normalise to the internal schema, and upsert into the database. Connectors exist for OpenStreetMap listings, Booking.com prices, festival and web events, heritage entry fees, Wikidata enrichment, Wikivoyage knowledge and curated notes. A pipeline script can run all sources, only the sources that are due, a single source or a single district, and supports a dry-run mode. An APScheduler job runs nightly at 02:30 Asia/Colombo time, with maintenance jobs for old sessions and travel-time cache. Administrators can also trigger synchronisation of listings or events from the admin API. New records are reviewed in the admin panel before travellers see them.

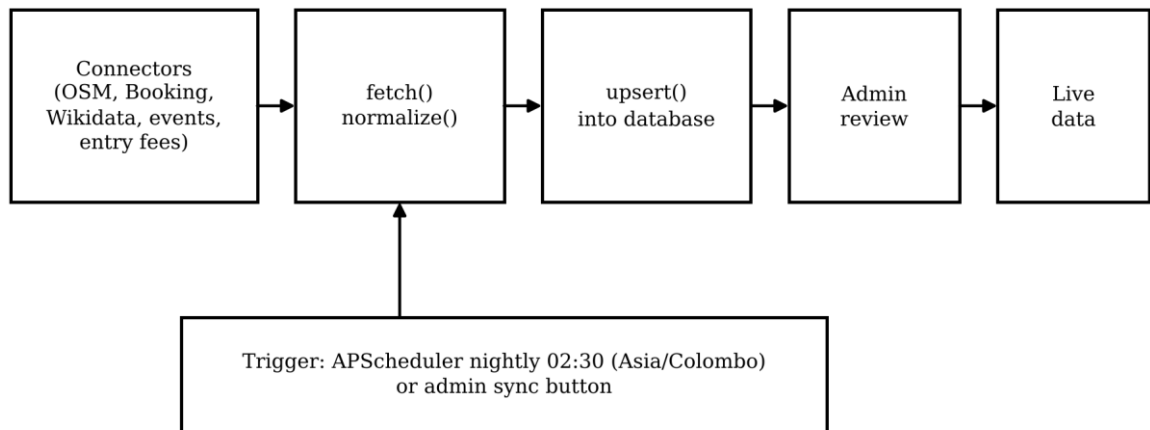


Fig. 8. Data ingestion and refresh flow.

At the time of writing the database holds 25 districts and about 6,500 listings with photographs where available.

5.6 Administration

The admin interface lets an administrator verify or reject listings and events, search and manage users, view usage statistics, and configure the AI model chain and API keys. Subscription revenue is reported as unavailable because billing was not built, rather than showing an invented figure.

5.7 Continuous Integration and Delivery

Each repository has a GitHub Actions workflow. The tests run on every push and pull request with no live services and only placeholder secrets. On the main branch, a successful test run builds container images and deploys them to the server with health checks and automatic rollback; the web application deploys to Vercel from its release branch. Because tests gate deployment, a commit that fails linting or tests never reaches production.

TABLE V
CI/CD PIPELINES

Repository	Test job	Deploy job
backend	npm ci, prisma generate, lint, unit tests, end-to-end tests	Build images to GHCR, SSH to EC2, run deploy script with health check and rollback
ai-backend	pip install, pytest excluding tests that need external services	Build image to GHCR, deploy to EC2
frontend-web	npm ci, lint, unit tests	Deploy to Vercel

6. Testing and Evaluation

6.1 Test Strategy

The strategy is to test every service and every boundary between services: browser to NestJS, NestJS to the AI service, the AI service to LLMs and external APIs, and NestJS to the database. A defect at a boundary is invisible to tests that only exercise one side. The philosophy is to write as many tests as pay for themselves, concentrating on ownership rules, failure paths and deterministic logic. Since LLM output is non-deterministic, correctness is defined as schema-valid, budget-respecting and policy-compliant output, together with a tested fallback when the model is unavailable.

TABLE VI
TEST LAYERS AND TOOLS

Layer	Tool	What it checks
AI service unit and integration	pytest (826 tests in 61 files; external calls mocked)	Graph routing, slot filling, scoring, clustering, itinerary, budget, output validator, fallback, follow-ups, RAG, tools, connectors, internal-token guard
API unit	Vitest with hand-written Prisma fakes	Ownership scoping, budget status thresholds, mappers, AI-client error mapping
API end-to-end	Vitest and supertest	Security cases SEC-01 to SEC-08 and access-control cases AC-01 to AC-09 through the real guard chain
Front-end unit	Vitest, React Testing Library	Route middleware, token refresh, trip store, key components
Browser journeys	Playwright with authenticated setup	Four journey tests and five accessibility tests through the real interface
Golden scenarios	scripts/e2e_check.py	Twelve end-to-end planning scenarios against the full stack
RAG quality	scripts/eval_rag.py, 25 golden questions	Retrieval hit rate, refusals and citations
Load	autocannon	Throughput and latency baseline

6.2 Unit and Integration Results

The automated suites run with no live services and no real API keys, and all executed tests pass in continuous integration. Statement coverage was measured at 80.1% for the NestJS API (713 of 890 statements) and 75.3% for the web front end; coverage is reported but not enforced as a gate, because a threshold on a young suite would fail the build for the wrong reasons. Mocked unit tests exposed one real defect during their creation: the event-search service dropped the from-date filter when both from and to dates were given. It was fixed and is now covered by a test.

6.3 Security and Access-Control Testing

The end-to-end tests send requests through the actual guard chain. They confirm that unauthenticated requests are rejected, that a traveller cannot call admin routes, and that one traveller receives 404 when requesting another traveller's itinerary, expenses or chat history. Unit tests assert that every service query receives the user identifier. The AI service has an internal-token check with its own tests. The documented residual risk is that the AI service relies on network isolation for its own exposure.

6.4 Golden Scenario Testing

Twelve scripted scenarios exercise the full stack, including a normal plan, budget limits, multi-destination trips, follow-up edits, weather questions and an LLM-unavailable case. In the recorded live run of 27 September 2026, 10 of 12 scenarios passed (83.3%), below the target of 11 of 12. The scenario with the LLM unavailable passed, returning a fallback plan with HTTP 200. The two failures are discussed in Section 6.7.

TABLE VII
GOLDEN-SCENARIO OUTCOME (27 SEP 2026)

Result	Count	Notes
Passed	10	Including LLM-unavailable fallback (HTTP 200, plan_source = fallback)
Failed	2	Day-2 cost not lower after a cheaper request; unhandled exception on a follow-up whose planner call fails

6.5 RAG Evaluation

The quality of the retrieval-augmented question-answering feature was measured with a golden set of 25 questions stored with the tests. Twenty of the questions can be answered from the knowledge base, and each is paired with the title of the source document that should be retrieved. The other five are deliberately out of scope, and the system is expected to refuse them rather than guess. The evaluation script, `scripts/eval_rag.py`, runs every question against the real database and embedding model and reports three measures: retrieval hit@5 (whether the expected source appears in the top five results), refusal accuracy for the out-of-scope questions, and the number of answers given without a citation. The targets set when the feature was built were a hit@5 of at least 80%, a refusal accuracy of 100% and no uncited answers. The script is re-run after any change to chunking, embeddings or retrieval.

In the retrieval evaluation recorded on 30 September 2026, the expected source appeared in the top five results for all 20 answerable questions, giving a hit@5 of 100% (20/20) and exceeding the 80% target. This shows that the hybrid search, which merges vector and full-text results by reciprocal rank fusion, finds the correct document both for questions that use exact names, such as "Dalada Maligawa" or "ETA", and for questions written in general terms.

That run measured retrieval only. Refusal accuracy and citation coverage need the full answer pipeline with a live language model, and they had not been measured at the time of writing. The answer stage is still protected by design: a minimum-score gate makes the system reply that it has no reliable information when the best retrieved passage scores below 0.55, and unit tests check both the refusal path and the parsing of citations with the model mocked. Because the golden question set was revised on 1 October 2026, after the recorded run, the full evaluation should be repeated before release to confirm all three targets.

6.6 Performance and Load

A single manual baseline was recorded with autocannon against the public listings route, using 20 connections for 30 seconds on a local stack. The server sustained an average of 394 requests per second with median latency 50 ms, 99th-percentile latency 68 ms and maximum 145 ms, about 12,000 requests in total. In practice the limiting factors are the quotas of external services (geocoding, routing and free-tier LLMs), which is why those calls are cached and why the fallback planner exists.

6.7 Defects and Limitations Found

- **Follow-up crash.** When a follow-up turn's planner call failed while an earlier itinerary existed, the verification node validated a missing planner output and raised an exception. The remedy is to guard the node on the planner output so the request degrades to the fallback plan. It was recorded as open on 27 September 2026 and remains open at the time of writing.
- **Cheaper day-2 request.** In one scenario the rebuilt day was not cheaper; this was treated as a data-coverage gap and not a flaky test.
- **Free-tier quotas.** Gemini and Groq limits restrict how often the full LLM path runs; the fallback planner covers the gap.

6.8 Accessibility and Browser Checks

The Playwright suite includes automated accessibility checks, and page-load behaviour was inspected with browser developer tools. These checks complement, but do not replace, the usability study in Chapter 7.

7. Usability Evaluation

7.1 Method

Besides measuring performance with testing tools (Chapter 6), the system was evaluated for usability with external users, to measure its user-friendliness and user experience. A survey was run among real users with the System Usability Scale (SUS), a ten-item questionnaire with five response options from strongly disagree to strongly agree [16]. Each answer is converted to a contribution of 0 to 4 (for odd-numbered statements the answer minus 1, for even-numbered statements 5 minus the answer), the ten contributions are added, and the sum is multiplied by 2.5 to convert the range 0–40 to 0–100. A score above 68 is considered above average and below 68 below average [17], [18].

To keep the results consistent, every user received the system link, the survey link and a short user guide with the same task list, covering planning a trip, reading and changing the plan, saving it, tracking the budget and exploring places. A pre-created traveller test account was provided so that no time was spent on registration; it gave no access to the database or to administrator functions. For each user the form recorded the ten SUS answers (1 to 5), whether all or most of the tasks were completed, and the total time taken. The answers were entered into a spreadsheet that converts each answer and calculates every participant's SUS score and the average. The statements used are listed in Appendix C.

7.2 Participants

The system was evaluated with 17 external users who were not members of the development team. Each completed the task list and the questionnaire once.

7.3 SUS Results

An average SUS score of 76.03 out of 100 was observed for SmartJourney, which is above the average benchmark of 68 and within about four points of 80.3, the level commonly regarded as excellent. The median was 75, the lowest score was 60 and the highest was 90. Fourteen of the 17 users (82.35%) scored above 68, and the three remaining scores were 60, 65 and 67.5.

TABLE VIII
SUS AND TASK RESULTS (17 PARTICIPANTS)

Measure	Value
Mean SUS score (0–100)	76.03
Median SUS score	75
Participants above 68	14 of 17 (82.35%)
Completed all tasks	12 of 17 (70.59%)
Completed most tasks	5 of 17 (29.41%)
Average completion time	11.71 minutes

7.4 Results by Statement

Table IX and Figs. 9 and 10 show how many users agreed or strongly agreed with each statement. The most positive results were for confidence (Q9) and ease of use (Q3), each agreed by 76.47% of users, followed by learning to use the system quickly (Q7) at 70.59%. Frequent use (Q1) and good integration of functions (Q5) were each agreed by 64.71%.

TABLE IX
AGREEMENT WITH EACH SUS STATEMENT

No.	Statement (short form)	Mean answer (1–5)	Agree / strongly agree	Agreement
Q1	Like to use frequently	3.88	11 / 17	64.71%
Q2	System is unnecessarily complex	1.76	1 / 17	5.88%
Q3	Easy to use	4.35	13 / 17	76.47%
Q4	Need technical support	2.12	3 / 17	17.65%
Q5	Functions well integrated	3.76	11 / 17	64.71%
Q6	Too much inconsistency	1.71	1 / 17	5.88%
Q7	Learn to use quickly	4.06	12 / 17	70.59%
Q8	System is cumbersome	2.06	1 / 17	5.88%
Q9	Felt confident using	3.94	13 / 17	76.47%
Q10	Needed to learn a lot first	1.94	1 / 17	5.88%

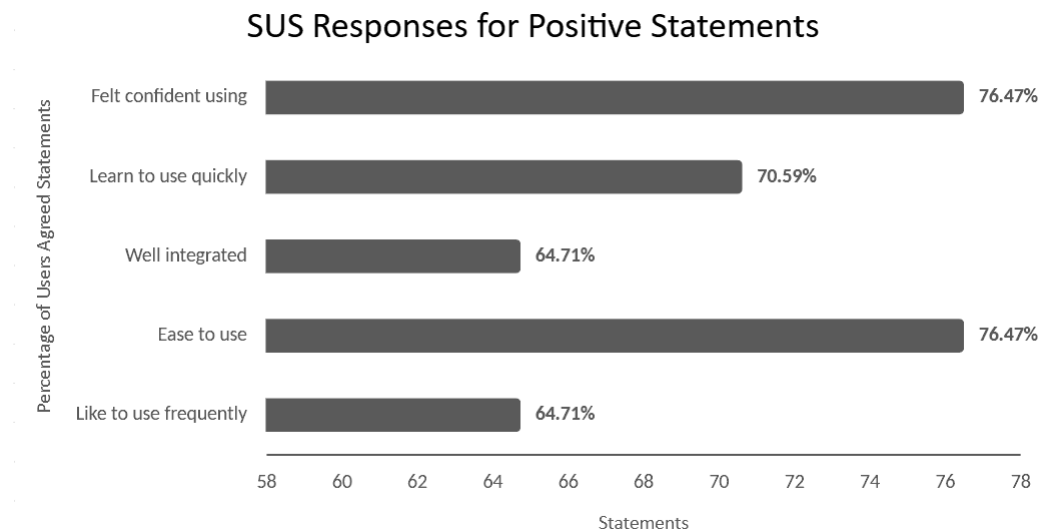


Fig. 9. SUS responses for positive statements (percentage of users who agreed).

For the negative statements, agreement should be low. Only one user (5.88%) agreed that the system was unnecessarily complex, inconsistent, cumbersome, or required a lot of prior learning. The highest agreement with a negative statement was for needing the support of a technical person (Q4), at three users (17.65%).

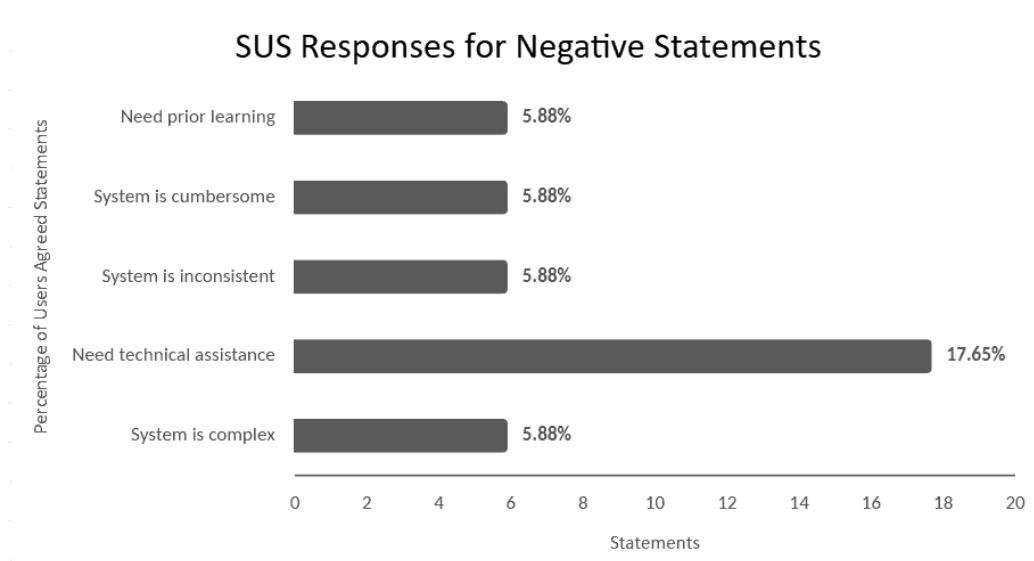


Fig. 10. SUS responses for negative statements (percentage of users who agreed).

7.5 Discussion

The results show that most users found SmartJourney easy to use, learned it quickly and felt confident with it, and almost none found it complex, inconsistent or cumbersome. All 17 users completed at least most of the tasks, and 12 completed all of them, in about 12 minutes on average. The weakest points were the perceived need for technical support (Q4), and the lower agreement with frequent use (Q1) and integration of functions (Q5). These suggest adding short in-app guidance for first-time users and making the links between planning, saving and budget tracking more visible. The small sample of 17 volunteers limits how far the results can be generalised.

8. Results and Discussion

8.1 Achievement of Objectives

TABLE X
OBJECTIVES AND OUTCOME

Objective	Outcome	Evidence
Functional, centralised platform	Achieved	Live site; Chapter 5
Analyse travel-planning challenges	Achieved	Chapters 1–2; proposal and feasibility study
Personalised itinerary generation	Achieved	Planning pipeline; golden scenarios 10/12
Agentic AI on curated data	Achieved	Verified-listing constraint; admin review; validator
Budget planning and optimisation	Achieved	Budget logic, budget tracker, cheaper follow-up
Evaluation with testing and users	Achieved: mean SUS score 76.03 from 17 users	Chapters 6–7

8.2 Strengths

- The system degrades gracefully: an unavailable LLM never produces a server error.
- Facts come from verified data, so the planner cannot invent places.
- Ownership scoping is applied everywhere and verified by tests.
- Data refresh is automated, with an administrator review step.
- Deployment is repeatable, gated by tests and reversible through health checks and rollback.

8.3 Limitations

- Subscription billing was not built.
- Free-tier LLM and API quotas limit throughput, and the full LLM path often falls back under load.
- Restaurants have no free photograph source, so a placeholder is shown.
- Google sign-in needs an OAuth client for each environment and was not configured; email and password sign-in always works.
- Load testing was a single local baseline and was not automated.
- Playwright journeys and golden scenarios are run manually before a release, not in CI.

8.4 Threats to Validity

The usability sample is small and consists of volunteers, who may be more technology-literate than typical travellers. AI output varies between runs, so two participants may see different itineraries for the same request. The study uses a controlled task list, so it does not measure long-term use.

9. Conclusion and Future Work

9.1 Conclusion

SmartJourney shows that a conversational, multi-agent planner can be made trustworthy by grounding it in verified local data, validating its output and providing a deterministic fallback. The delivered system includes a deployed web application, an administrator interface, a LangGraph planning service, a RAG knowledge base, automated data ingestion and a CI/CD pipeline that gates releases on tests. Testing covers all service boundaries, and a usability study using the SUS provides evidence from real users.

9.2 Future Work

- Fix the open follow-up defect and raise the golden-scenario pass rate to the target.
- Move Playwright journeys and golden scenarios into scheduled CI runs.
- Use a paid LLM tier or self-hosted models for a more reliable planner path.
- Add automated load tests with mocked external services and set pass/fail thresholds.
- Expand data coverage, including restaurant imagery and more events.
- Add secure service-to-service authentication so the AI service no longer depends only on network isolation.

References

- [1] Google, "Google Travel." [Online]. Available: <https://travel.google.com/>. (Accessed: 4 Jul. 2026).
- [2] TripAdvisor LLC, "TripAdvisor." [Online]. Available: <https://www.tripadvisor.com/>. (Accessed: 4 Jul. 2026).
- [3] Booking.com B.V., "Booking.com." [Online]. Available: <https://www.booking.com/>. (Accessed: 4 Jul. 2026).
- [4] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in Proc. Int. Conf. Learn. Represent. (ICLR), 2023.
- [5] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, 2020.
- [6] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in Proc. 32nd Int. ACM SIGIR Conf., 2009, pp. 758–759.
- [7] LangChain, "LangGraph documentation." [Online]. Available: <https://langchain-ai.github.io/langgraph/>. (Accessed: 3 Oct. 2026).
- [8] Vercel, "Next.js documentation." [Online]. Available: <https://nextjs.org/docs>. (Accessed: 4 Jul. 2026).
- [9] NestJS, "NestJS documentation." [Online]. Available: <https://docs.nestjs.com/>. (Accessed: 4 Jul. 2026).
- [10] S. Ramírez, "FastAPI documentation." [Online]. Available: <https://fastapi.tiangolo.com/>. (Accessed: 3 Oct. 2026).
- [11] PostgreSQL Global Development Group, "PostgreSQL documentation." [Online]. Available: <https://www.postgresql.org/docs/>. (Accessed: 4 Jul. 2026).
- [12] pgvector contributors, "pgvector: open-source vector similarity search for Postgres." [Online]. Available: <https://github.com/pgvector/pgvector>. (Accessed: 3 Oct. 2026).
- [13] Keycloak, "Keycloak documentation." [Online]. Available: <https://www.keycloak.org/documentation>. (Accessed: 3 Oct. 2026).
- [14] Prisma, "Prisma documentation." [Online]. Available: <https://www.prisma.io/docs>. (Accessed: 3 Oct. 2026).
- [15] Google, "Gemini API documentation." [Online]. Available: <https://ai.google.dev/gemini-api/docs>. (Accessed: 4 Jul. 2026).
- [16] J. Brooke, "SUS: A 'quick and dirty' usability scale," in Usability Evaluation in Industry, P. W. Jordan et al., Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.

- [17] J. Sauro, "Measuring usability with the System Usability Scale (SUS)," MeasuringU, 2011. [Online]. Available: <https://measuringu.com/sus/>. (Accessed: 3 Oct. 2026).
- [18] J. Brooke, "SUS: A retrospective," J. Usability Stud., vol. 8, no. 2, pp. 29–40, 2013.

Appendix A: Deliverables and Where to Find Them

TABLE XI
SUBMISSION MAP

Requirement	Location
Source code	ai-backend/app, backend/src, backend/db, backend/prisma, frontend-web/src
Test scripts	ai-backend/tests, scripts/e2e_check.py, scripts/eval_rag.py; backend/src/**/*spec.ts, backend/test; frontend-web/src/**/*__tests__, frontend-web/e2e
Build scripts	Dockerfiles, package.json scripts, requirements.txt, docker-compose.yml, backend/deploy scripts, .github/workflows/ci.yml
Configuration	*.env.example, compose.prod.yml, Caddyfile, Keycloak realm export, vercel.json, tool configuration files
User manuals	Documentation/User Manual
Executable version	Live site: https://aismartjourney.vercel.app , or a local Docker run with the bundled demo data
Earlier documents	Proposal, feasibility report, schedule, SRS, design document and test plan in Documentation

Appendix B: Automated Test Inventory

TABLE XII
TEST INVENTORY BY REPOSITORY

Repository	Runner and command	Files
ai-backend	pytest: python -m pytest -m "not external" -q	61 test files, 826 tests
backend	Vitest: npm test; npm run test:e2e	20 unit spec files (160 tests), 3 end-to-end files (24 tests)
frontend-web	Vitest and RTL: npm test; Playwright: npm run test:e2e	5 spec files (18 tests); 2 Playwright files (4 journeys, 5 accessibility checks)

Two live checks run outside these runners: `scripts/e2e_check.py` for the twelve golden scenarios and `scripts/eval_rag.py` for the 25 RAG questions. All three automated suites were re-run on 3 October 2026 and every test passed.

Appendix C: Usability Questionnaire

Participants rate each statement from 1 (strongly disagree) to 5 (strongly agree).

TABLE XIII
SUS STATEMENTS

No.	Statement
1	I think that I would like to use this system frequently.
2	I found the system unnecessarily complex.
3	I thought the system was easy to use.
4	I think that I would need the support of a technical person to be able to use this system.
5	I found the various functions in this system were well integrated.
6	I thought there was too much inconsistency in this system.
7	I would imagine that most people would learn to use this system very quickly.
8	I found the system very cumbersome to use.
9	I felt very confident using the system.
10	I needed to learn a lot of things before I could get going with this system.

Appendix D: Team Contributions

Hewawasam H.H.S.T.T. (230247T)

AI service. Built the core LangGraph planning pipeline, including:

- tools for weather and disaster alerts (EONET, USGS, GDACS), geocoding and routing, with caching
- multi-turn sessions and slot filling
- the policy guard
- geographic day clustering and cross-day variety
- budget limits and estimated trip cost
- multi-place trips
- the Gemini/Groq provider chain and admin-managed LLM configuration
- the move from Supabase to PostgreSQL

Also fixed the data connectors and Wikidata enrichment, and wrote the test suites for prompts, scoring, fallback validation, RAG and the scrapers.

API. Set up the NestJS project, PostGIS and Docker Compose, and built the explore, budget and trip modules, the AI-service integration for chat, entry-fee management, the Prisma 7 upgrade, and unit tests for the services and guards.

Web front end. Created the Next.js project and built the landing page, the chat, explore, saved-itineraries and budget-tracker pages, the multi-day route map, the citation sources list, the admin entry-fee panel, and the Vitest and Playwright setup.

Deployment. Set up CI/CD with GitHub Actions and GHCR, the AWS EC2 server with health checks, rollback and release scripts, the Vercel deployment, the Keycloak login theme and the production setup guides.

Jayasinghe E.A.T.D. (230279R)

AI service. Wrote the first planning and recommendation agents with their prompts, the database tool and the shared trip state. Also built:

- the first RAG version (embeddings and retrieval)
- OpenStreetMap (Overpass) data ingestion
- a Wikipedia popularity connector to improve listing scores
- informational follow-ups and targeted "make day N cheaper" re-planning
- the budget breakdown and default price-level estimates
- category exclusion and the start location set by the user
- Sri Lanka standard dates throughout
- internal-token authentication between the API and the AI service

Identity and API. Set up Keycloak authentication (realm and roles, JWT verification, user provisioning, Google sign-in linking, and SMTP for verification and password-reset emails). Built the admin content and user-management services with bulk verification of listings and events, user preferences and dynamic conversation titles, and the email notification system for trip reminders, weather alerts and budget alerts. Wrote the end-to-end security and access-control tests and normalised line endings in the migration checksums.

Web front end. Built the next-auth integration with Keycloak (sign-in, federated logout, password reset), account and travel-preference settings, the admin dashboard (moderation, bulk approval, user management, analytics, content forms), geolocation and district suggestions in the chat, the resizable split pane, notification settings, and the Playwright end-to-end and axe-core accessibility tests.